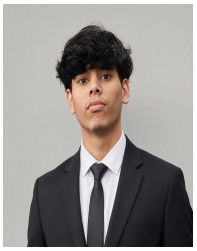
PHASE-II: PROJECT AND TEAM INFORMATION

Project Title

RESQ — Intelligent Disaster Rescue & Emergency Response Simulator. ***"Algorithms That Save Lives."***

Student / Team Information

Team: RESQ Force ID: DSCPP-III-2026-T117		*Photo*
Team member 1 (Team Lead)	*Vanya Rathi – 2510383410* *2510383410@geu.ac.in* *University Roll No.: 2028514* *Phone No.: 9259011570* *Section: DS-2*	
Team member 2	*Akshat Dobhal – 2510370353* *2510370353@geu.ac.in* *University Roll No.: 2028631* *Phone No.: 9389135760* *Section: ML-2*	
Team member 3	*Ayush Kandwal – 2510380235* *2510380235@geu.ac.in* *University Roll No.: 2028405* *Phone No.: 9068120181* *Section: ML-4*	
Team member 4	*Nitin Singh – 2510370229* *2510370229@geu.ac.in* *University Roll No.: 2028786* *Phone No.: 6395245688* *Section: ML-1*	

Project Mentor: *Dr. Prabhdeep Singh* ***Team ID:*** *DSCPP-III-2026-T117*

IMPLEMENTATION AND PROGRESS

Project Summary and Phase-II Objective

RESQ is our class project on one simple question: when many people need help at once, who do we reach first, and by which road? In Phase I we only had a plan on paper, a C/C++ design built around data structures and classes. In Phase II our job was to make it actually run. So we built victim entry, a queue that puts urgent cases on top, routing over a disaster map, and a way to track each rescue. We also made a browser version so the decisions show up on a map and a dashboard. To be clear, this is a learning model. Nobody should use it for real emergency calls.

Work Completed Since Phase I

Right now the web version lets us add victims, see them waiting in order of priority, switch disaster settings and look at a weighted map of places in Dehradun. Whoever plays the dispatcher picks a request, sends a free rescue unit and sees the route it will take. The dashboard keeps four counts apart: registered victims, waiting requests, active operations and finished rescues. You can filter all of it by priority.

One thing we want to be honest about: the C/C++ code from earlier and the web demo are two separate builds. The browser runs the logic in TypeScript, with React for the screens. It does not call our C/C++ classes. We point this out because our mentor will ask how the course topics fit in, and we do not want to overclaim.

Data Structures, Algorithms and Class Design

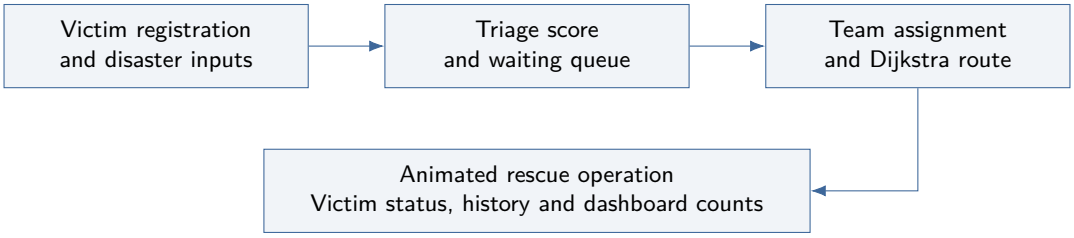
Queues. *In the C version, normal calls go into a plain FIFO queue. In the web demo, requests are ranked by how serious they are and how long they have waited. For this we used a binary max-heap, so a bigger triage score means the case comes out first. Our Phase-I proposal said "min-heap" in one place. Both work, as long as the score and the comparison point the same way.*

Map and routes. *Every location is a node and every road is an edge with a cost. Dijkstra finds the cheapest path and skips roads we have marked as blocked. BFS tells us which areas can still be reached at all. Costs never go below zero, since Dijkstra breaks with negative weights.*

Records and history. *The linked list, FIFO queue and stack live in the C code. On the web side we just used normal JavaScript arrays and objects for records and history, so we don't call those pointer-based C structures.*

Classes. *The C++ side has Victim, RescueTeam, Ambulance, Hospital, Disaster, Location and RescueOperation. Data is kept private behind member functions (encapsulation). Ambulance and RescueTeam come from a shared base class and override its dispatch function, which is where inheritance and polymorphism come in. Joining this native code with the web version is still left to do.*

System Architecture and Rescue Workflow



A new request gets saved and dropped into the waiting queue. The dispatcher picks a victim and a team that is free. The routing part works out a path on the road map as it is at that moment. The unit then moves along that path on screen, and when it arrives the victim's status and the dashboard numbers change. If there is no path at all, the request just stays pending. We made sure it never shows a rescue that didn't happen.

EVALUATION, CONTRIBUTIONS AND NEXT STEPS

Initial Evaluation and Test Plan

At this stage we are checking that the prototype behaves the same way every time. We are not saying it cuts real rescue times. Our first fixed tests passed. The heap gave back RED, then YELLOW, then GREEN. Dijkstra gave costs of 5, 6 and 7 from CMD to Market, Clock and Fire. Keep in mind these are map costs we made up, not real minutes. The table below is the checklist we are working through. We still have to run a fair comparison against plain first-come-first-served dispatch, with bigger loads, before we can talk about any speed-up.

Case	**What should happen**
Mixed urgency	Higher-score victims come before lower-score ones. We still need to write down a rule for ties.
Open and blocked roads	Dijkstra picks the cheapest open road. Blocking a road should change the route or make the place unreachable.
No free team / no route	The victim is not marked as dispatched or rescued. The request stays in the queue.
Rescue completion	Victim status, team availability, the operations list and the rescue count all agree.
Empty queue / reset	Nothing gets dispatched by mistake, and all counts and lists go back to a clean start.

Individual Contributions

We kept the same split of work we showed in the Phase-I presentation. It tells you who owns which part.

Vanya Rathi (DS-2): *Architecture, triage logic, integration and documentation.*

Akshat Dobhal (ML-2): *Linked list, queue, priority queue and stack modules.*

Ayush Kandwal (ML-4): *Graph module, BFS and Dijkstra routing.*

Nitin Singh (ML-1): *C++ classes, scenario testing, dashboard and reports.*

Limitations and Remaining Work

Road costs, triage scores and rescue times are all made up for the simulation. There is no live GPS, no link to any emergency service, and our triage score is not a medical standard. The web demo also doesn't prove the C/C++ code is connected, and it doesn't save data for many users. We still need to write a tie-break rule, see what happens when a road gets blocked during a rescue, test hospital and ambulance allocation, and check that history is saved properly. And we need to run FIFO and priority dispatch side by side, with the same victims and the same teams, before we say anything about response time.

Next Milestones and Phase-II Deliverables

Next up: *Finish automatic tests for edge cases and compare the C modules with the browser logic. After that, run the whole flow from dispatch to rescue for earthquake, flood and fire, note down results we can repeat, and finish the user guide and class diagram.*

What we have now: *The RESQ web prototype, the earlier C/C++ source package, a priority queue and route map you can see working, the operations dashboard, and this report. At the next review we want to show what has been tested and what is still left, instead of quoting a completion percentage we never measured.*

References

1. *Cormen et al., Introduction to Algorithms, 4th ed., MIT Press, 2022: heaps, BFS and shortest paths.*
2. *E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, 1, 269–271, 1959.*
3. *B. Stroustrup, The C++ Programming Language, 4th ed., 2013; Kernighan and Ritchie, The C Programming Language, 2nd ed., 1988.*
4. *NDMA, India: disaster-management guidance,* `https://ndma.gov.in`*. Used for background only; RESQ is not an approved emergency tool.*